Seven tips for working with AI

A field-tested playbook,

illustrated with a forecasting experiment we ran in this unit.

JRC C4 · Dermot O'Brien · 2026

TIP 1

Learn how to program first

- AI is a **multiplier of what you already know.**
- It cannot replace the domain knowledge that an economist or engineer in C4 brings to a forecasting model, an impact assessment, or a policy briefing.
- If you don't know what "good" looks like for the task, no prompt will rescue you — you won't be able to tell when the output is wrong.

The rest of this talk is about how to convert your domain knowledge into AI output that's actually usable.

TIP 2

Be as specific as humanly possible

> AI is only as good as the context you give it.

- Most people don't give it enough.
- The next slides show **an experiment we ran in this unit** to quantify how much it matters.

The experiment

- **Task**: forecast 168 hours (one week) of hourly German electricity load, 2020-01-01 to 2020-01-07.
- **Data**: Open Power System Data (OPSD) / ENTSO-E, 2015–2020, hourly.
- **Agent**: Claude Code, Opus 4.7 — same agent in every run.
- **Three prompts**, three workspace setups:

Level 1 — beginner

"Forecast first week of January 2020 German hourly electricity load."

10 words · no AGENTS.md

Level 2 — average

Names the data file, the horizon, the libraries. Asks for a plot and an accuracy number.

46 words · 7-line AGENTS.md

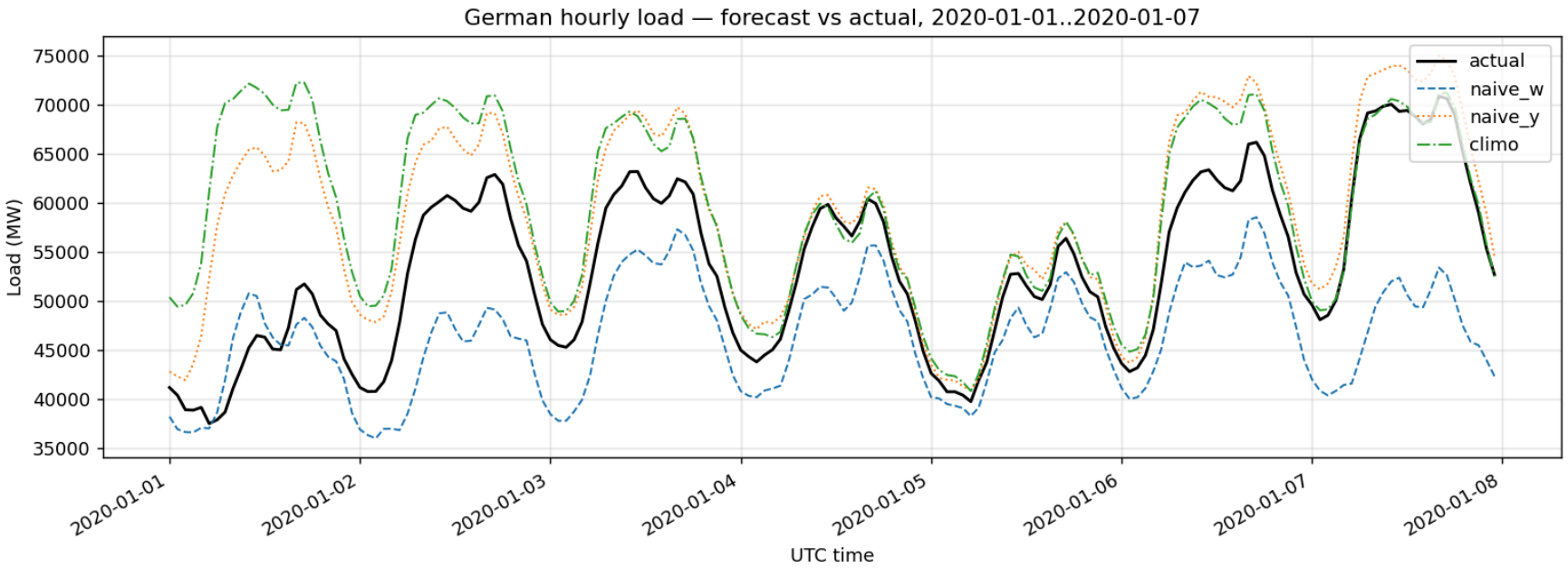
Level 3 — research-grade

Structured prompt (TASK / BACKGROUND / DO NOT). 6-model bake-off. Validation splits. Calibrated intervals. Written methods.

1,673 words · 113-line AGENTS.md

Level 1 — what 10 words gets you

> Forecast first week of January 2020 German hourly electricity load.



Three baselines compared; L1 picked the best of three: **seasonal-naive (lag-364d), MAPE 10.76%**.

No validation set. No prediction intervals. No methodology document.

Level 2 — the prompt and the workspace

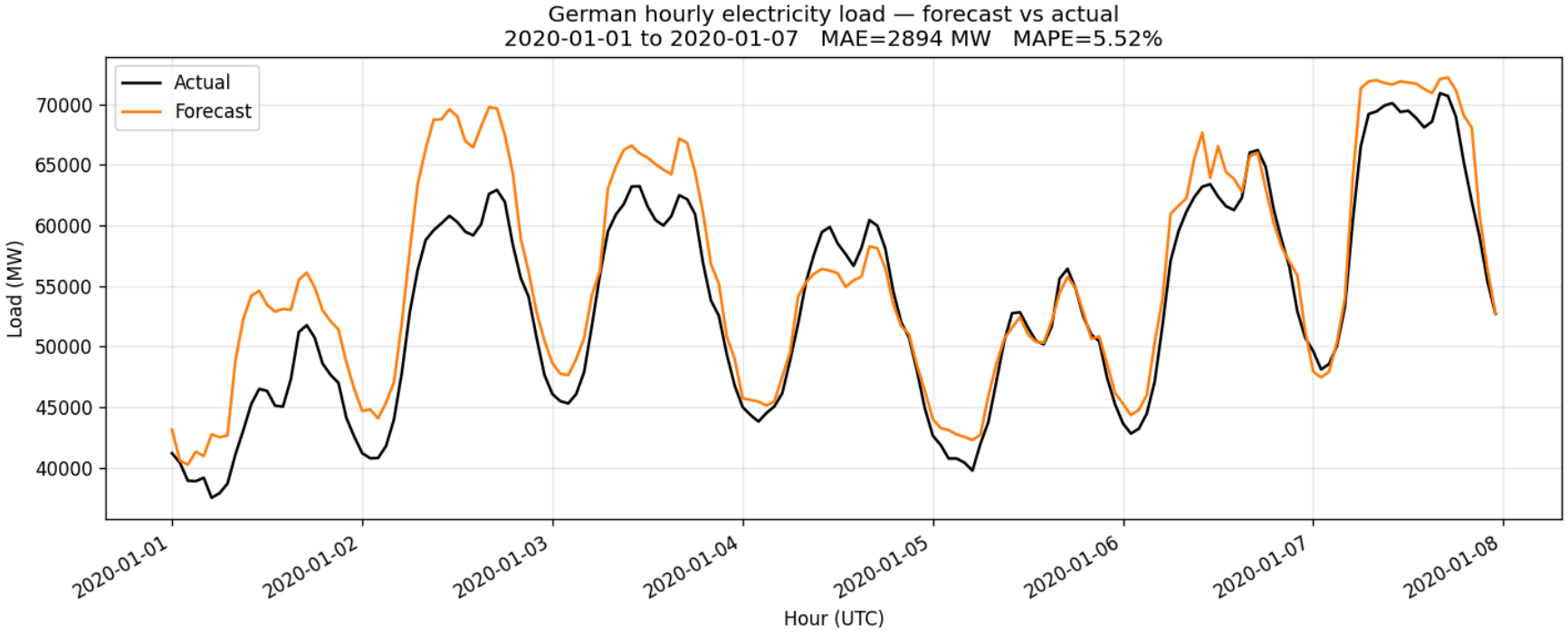
The prompt (46 words)

```
Build me a Python script that
forecasts German hourly electricity
load for the first week of January
2020. Train on the historical data,
plot it against the actual values,
and tell me how accurate it was.
```

AGENTS.md (7 lines)

```
# Notes for the AI
Python project. The file
opsd_de_load.csv has hourly
German load (MW) from OPSD.
Use pandas. Plot with matplotlib.
```

Level 2 — the result



The agent picked **GradientBoostingRegressor** with hand-engineered features (hour, day-of-week, lag-168h, lag-8760h).

MAPE 5.52%. Still no held-out validation, no prediction intervals, no methodology document.

Level 3 — the prompt structure (1,673 words)

TASK

Forecast 168 hours. Six-model bake-off. Report MAPE, RMSE, MAE.

BACKGROUND

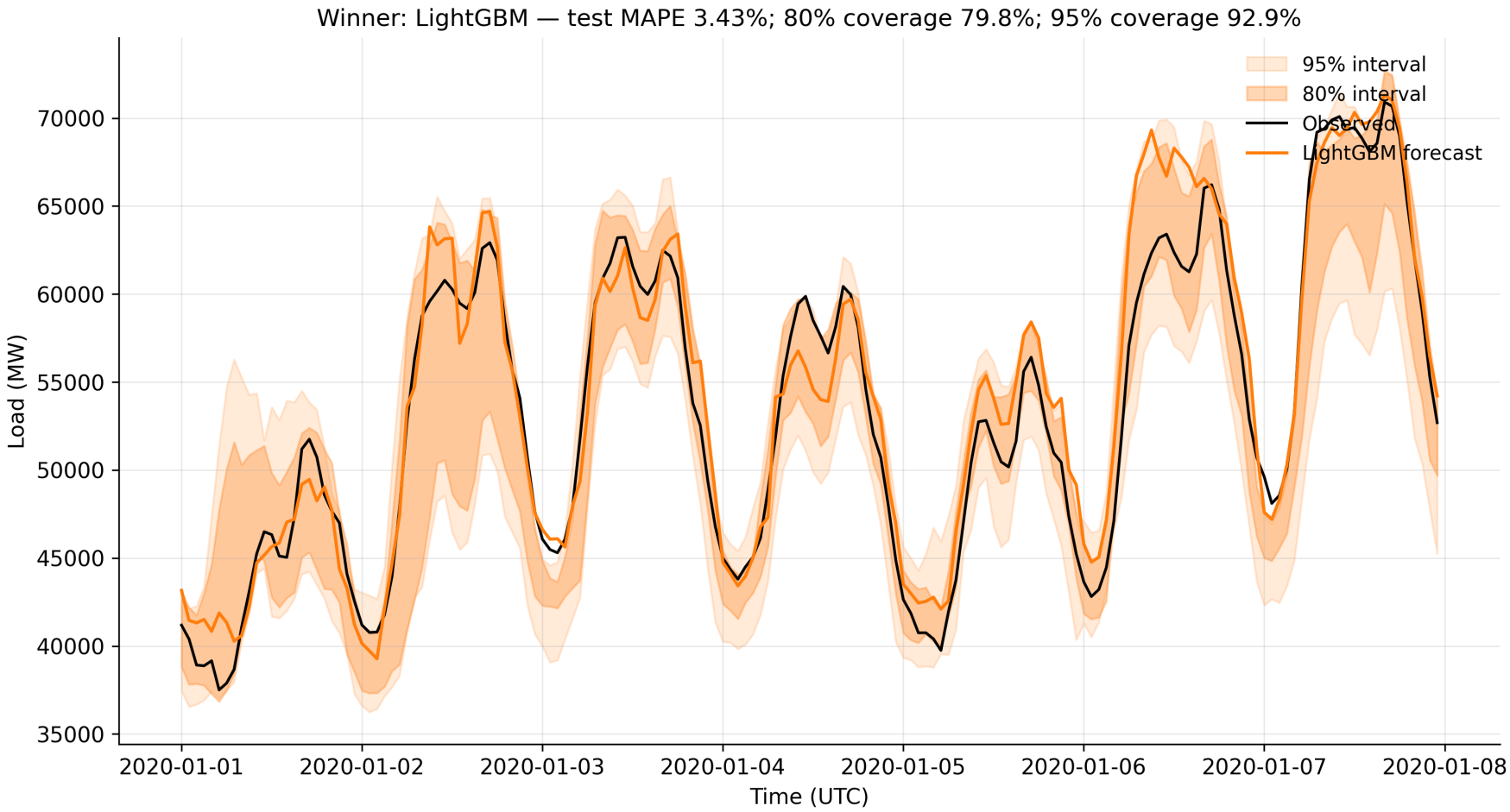
Data · Allowed inputs · Train/validate/test splits · Metrics · Required figures · Output structure · AGENTS.md schema (113 lines)

DO NOT

Use the test window for fitting. Pull external data. Skip refit-on-train+val. Silently drop missing hours.

Three sections. Same pattern for forecasting, paper drafting, literature search, debugging.

Level 3 — the result



LightGBM won a six-model bake-off. **MAPE 3.43%.**

80% prediction interval covered **79.8%** of points; 95% covered **92.9%** — essentially nominal.

The headline isn't the MAPE — it's everything around it

L1 — beginner

MAPE 10.76%

3 baselines, picked best

No validation set

No prediction intervals

No methods document

1 figure

Single script

L2 — average

MAPE 5.52% *(one model — got lucky)*

1 model:
GradientBoostingRegressor

No validation set

No prediction intervals

No methods document

1 figure

Single script

L3 — research-grade

MAPE 3.43% *(winner of a 6-model search)*

6 models compared systematically

Held-out validation set (2019 Q4)

80%/95% intervals —
79.8%/92.9% coverage

Methods document (transcript.md)

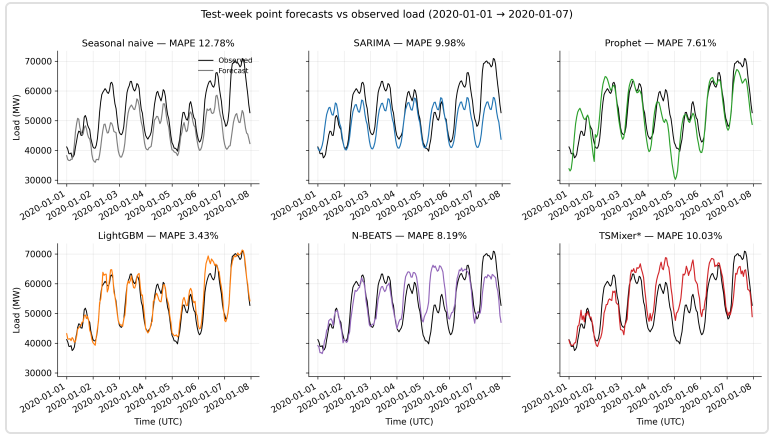
8 figures

Orchestrator + per-model modules

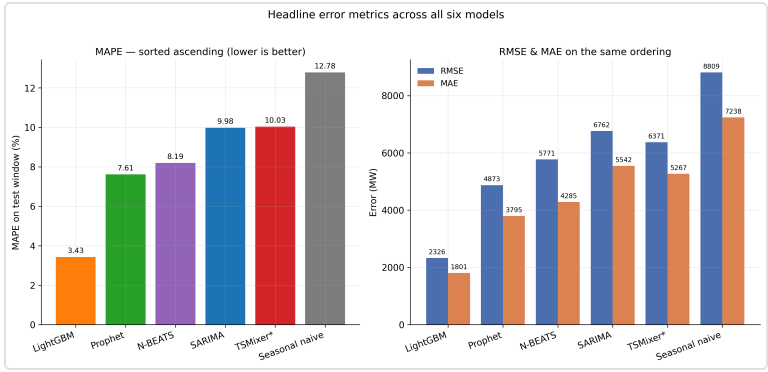
L2 might have got lucky picking a decent model.

L3 didn't have to be lucky — it searched, validated, calibrated, and documented.

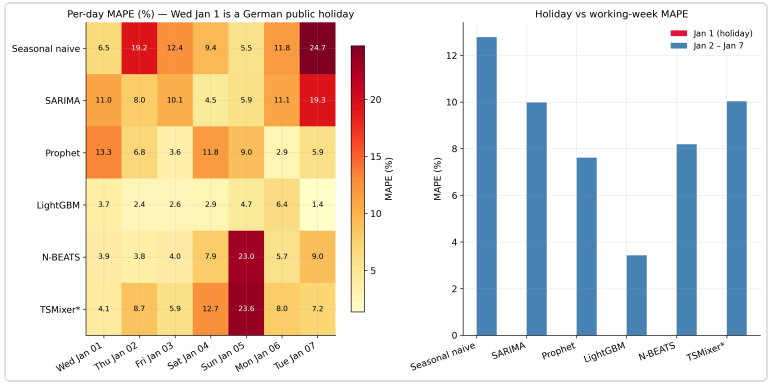
Some of what L3 actually produced



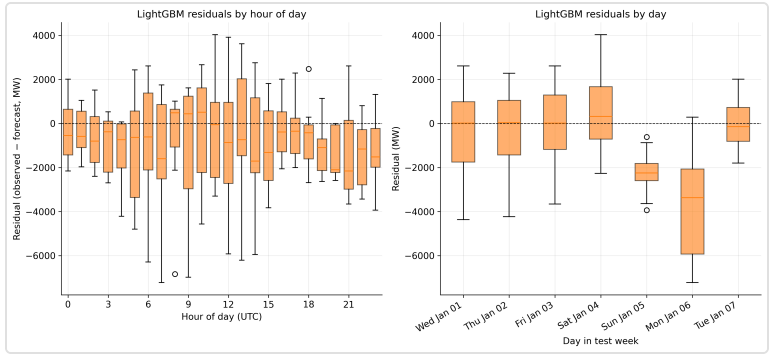
Six-model forecast comparison



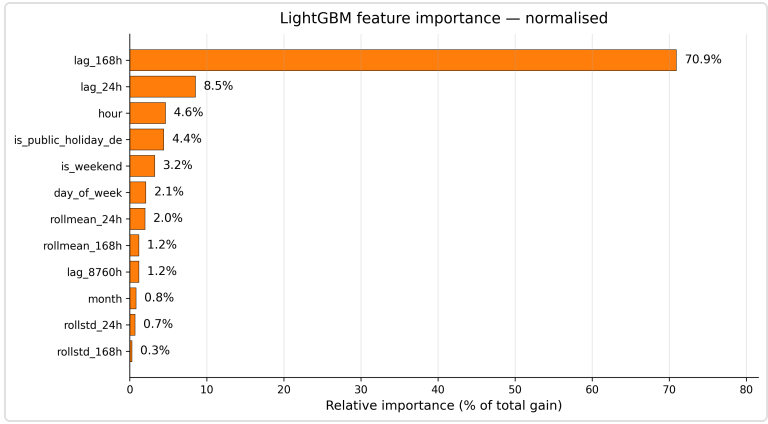
Scoreboard — MAPE, RMSE, MAE



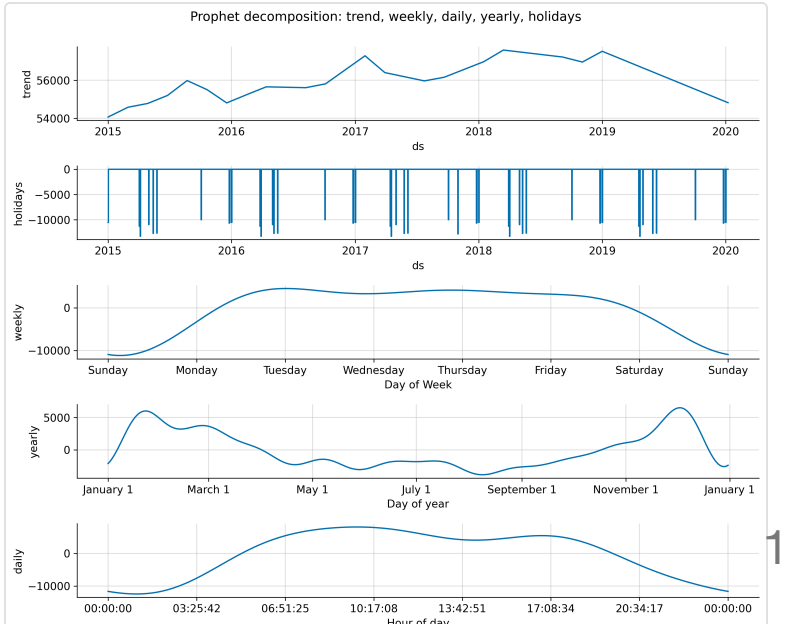
Per-day MAPE heatmap



Winner residuals by hour and day



LightGBM feature importance (normalised)



TIP 2 · SUB-TIP

Feed the AI what you'd Google

- AI assistants can read the web. Don't make them guess what you already know how to look up.
- **Paste the docs.** API references, methodology papers, code examples — drop them into the prompt or attach them as files.
- **Use `llms.txt` pages.** Some libraries publish their docs in a format designed for LLMs; reference the URL.
- **Use screenshots.** Easier than describing a chart you want or a UI layout you have in mind.

This is the easiest 20-minute upgrade you can make to your prompts.

TIP 2 · SUB-TIP

Have the AI improve your own prompt

1. Write a rough draft prompt with all the technical information you have.
2. Paste it back to the AI and ask: "*Rewrite this prompt using LLM best practices for a structured task description.*"
3. Review the rewrite. Keep what improves it, discard what's wrong.

A 30-second meta-step. Catches the structure-and-edge-case gaps you didn't think of.

TIP 3

The smaller the task, the better the results

- AI is good at small tasks. Worse at big, complex ones.
- Break a big task into smaller ones.
- **If you can't break it down, you don't understand the problem well enough yet.**
- This isn't an AI trick — it's fundamental engineering: plan the solution, decompose it, then code (or have AI code) each piece.

We did exactly this in our experiment — split into Phase 1 (run the three levels), Phase 2 (write the deck), with a plan-and-review step at every boundary.

TIP 3 · SUB-TIP

Let AI type for you — not think for you

- Letting AI **type** the code or the prose for you is fine, even excellent.
- Letting AI **think** for you means you stop applying the skills that justify your role.
- The moment you outsource the decisions, the model's mistakes become invisible to you — because you've stopped having an opinion.

Plan the solution yourself. Read the output critically. Push back when it's wrong.

TIP 4

Tell the AI what you *don't* want

The single highest-leverage structural change to any prompt is to add a "DO NOT" block. The pattern:

TASK
What to do, precisely — the objective, the metric, the output shape.

BACKGROUND
The data, the files, the constraints, the references the model needs.

DO NOT
What to avoid — common mistakes, things that look right but aren't.

Our Level 3 prompt had nine specific don'ts. The one that mattered most:
do not use the test window for fitting.

TIP 5

Tell the AI to remember — `AGENTS.md`

- Every Claude Code session reads an `AGENTS.md` file at startup, if present.
- Put project standing context there **once**, not in every prompt: tech stack, file layout, data sources, conventions, things the agent should never do.
- Our experiment quantified the gap:

L1

No AGENTS.md.

L2 — 7 lines

Data file location, libraries.

L3 — 113 lines

Allowed/forbidden inputs · output schema · `metrics.json` contract · DO-NOT block · style rules.

Single biggest leverage point most users miss.

TIP 5 · SUB-TIP

Have the AI draft `AGENTS.md` itself

1. Point the agent at your existing codebase: "*Analyse this repository and propose an AGENTS.md.*"
2. The agent reads the structure, infers conventions, drafts a v1.
3. You edit. Add the things it missed, remove the things it inferred wrongly.

Ten minutes' work. Saves hours of re-explaining your project for the rest of the year.

TIP 5 · SUB-TIP

Reusable workflows — Skills

- A **skill** is a named, reusable workflow you can invoke from the agent — a markdown file the agent reads when triggered.
- Lives in `~/.claude/skills/<name>/SKILL.md` (global) or `.claude/skills/<name>/SKILL.md` (project).
- We used one to build this deck: `/review-loop plan` / `/review-loop code` — codifies the plan-then-implement-then-review pattern.
- Heuristic: any workflow you do more than twice should be a skill. Sharable — drop a skill folder into a colleague's `.claude/skills/`.

AGENTS.md gives the agent persistent *context*. Skills give it persistent *procedures*.

TIP 6

MCPs can extend what AI can do

MCP = Model Context Protocol. Plug-in tools the agent can invoke at runtime.

A handful that are immediately useful for research code in C4:

- **Context7** — fetches up-to-date library documentation on demand. No more pasting docs into prompts.
- **Hugging Face MCP** — search models, datasets, papers; pull metadata.
- **GitHub MCP** — read repos, issues, PRs; useful when working across projects.

Find the MCPs that match your tech stack — there's probably one for every external system you touch.

TIP 7

Always give the AI a way to verify its work

AI should never just write code. It also needs a way to **prove the code works.**

- **Tests** — unit, integration, regression.
- **A held-out validation set** for any model fitting.
- **Calibration metrics** — prediction interval coverage, residual diagnostics.
- **A written discussion of failure modes** — where it expects the model to break.

> Our Level 3 prompt asked for all four. The 80%/95% prediction intervals came back covering **79.8%** and **92.9%** of observed points — essentially nominal coverage.

TIP 7 · SUB-TIP

Have the AI generate the tests too — but verify them

- The fastest way to get test coverage is to ask the agent to write the tests.
- The risk: AI-written tests can be tautological — they "pass" because they test the wrong thing.
- **Read every test it writes.** A failing test that catches a real bug is worth ten passing tests that don't.

Same principle for AI-written validation scripts, AI-written sensitivity analyses, AI-written everything. Trust, then verify the trust.

Closing thesis — AI amplifies the habits you already have

The seven tips are not really AI tips. They are **good engineering habits**:

- Be specific. Break the problem down. Tell collaborators what not to do. Write things down. Build verification in.

If you bring those habits, AI multiplies your throughput substantially.

If you don't — if you skip tests, skip docs, skip the "what could go wrong" question — AI amplifies **that** instead, faster than you can audit.

Be the kind of researcher whose habits AI is worth amplifying.

Your goal is leverage.

AI scales execution. You bring the judgement.

code & data: github.com/DermotOBrien-EC/LLM_coding_practical_example_JRC_LEGENT_presentation_15_05_26